

Нормализация и схема

Исходные данные были денормализованы: в одной строке одновременно находились сведения о заказе, пользователе, курьере, магазине, товаре, оплате, отмене и доставке. Такой формат удобен для выгрузки, но неудобен для хранения и аналитики в реляционной БД.

В исходном наборе наблюдались типичные проблемы денормализованных данных:

- повторение телефонов пользователей и курьеров во множестве строк;
- повторение адресов магазинов и названий товаров;
- смешение различных сущностей в одной таблице;
- невозможность удобно хранить историю смены курьера;
- повторение одного и того же товара в рамках одного заказа;
- наличие замен товара без надежного line-level идентификатора.

Для проектирования основной модели была выбрана третья нормальная форма (3NF). Этот выбор был сделан по следующим причинам:

- устраняет избыточность описательных атрибутов;
- уменьшает риск аномалий вставки, удаления и обновления;
- позволяет разделить независимые сущности на отдельные таблицы;
- хорошо подходит для дальнейшего построения витрин и ETL-процессов.

Функциональные зависимости

По данным были подтверждены зависимости:

user_id -> user_phone

driver_id -> driver_phone

store_id -> store_address

item_id -> item_title, item_category

Также было выявлено:

- один заказ может иметь нескольких курьеров;

- один и тот же товар может встречаться в заказе более одного раза;
- поле `item_replaced_id` указывает на замену товара, но не позволяет надежно восстановить полную бизнес-логику замены на уровне строки.

Декомпозиция

Для устранения избыточности и разделения сущностей была выполнена декомпозиция исходной таблицы на справочники и факт-таблицы.

Справочники	Факты
• <code>dim_city</code> • <code>dim_category</code> • <code>dim_payment_type</code> • <code>dim_cancellation_reason</code> • <code>dim_user</code> • <code>dim_driver</code> • <code>dim_delivery_address</code> • <code>dim_store</code> • <code>dim_product</code>	• <code>fct_order</code> • <code>fct_order_driver</code> • <code>fct_order_line</code>

Такой подход позволил:

- отделить пользователей, курьеров, товары, магазины и адреса в самостоятельные сущности;
- сохранить заказ как отдельное бизнес-событие;
- вынести строки заказа в отдельную таблицу;
- сохранить историю назначений курьеров;
- подготовить корректную основу для аналитической витрины.

Особое внимание было уделено двум сущностям:

- `fct_order_line`.

Ключ (`order_id`, `item_id`) не подходит в качестве уникального, так как один и тот же товар может встречаться в заказе несколько

раз. Поэтому строка заказа была выделена в отдельную факт-таблицу с суррогатным ключом `order_line_id`, а уникальность исходной строки обеспечивается парой (`source_file_name`, `source_row_number`).

- `fact_order_driver`.

Так как в исходных данных один заказ может быть связан с несколькими курьерами, хранить `driver_id` непосредственно в заказе было бы некорректно. Выделение отдельной таблицы позволило сохранить историю назначений курьеров и в дальнейшем использовать эти данные в аналитике.

- Адреса доставки и магазинов не были дополнительно нормализованы до уровня улицы, дома и квартиры, так как исходные данные представлены свободным текстом и не дают гарантии устойчивого разбора. Поэтому город был вынесен в отдельный справочник, а полный адрес сохранен как текстовый атрибут.

Ниже представлена нормализованная реляционная схема базы данных, построенная по результатам декомпозиции исходного набора данных.

Слой `core` реализован в 3NF, `stage` используется для сырой загрузки, `mart` — для итоговой аналитической витрины.

stage.raw_delivery_lines		
source_file_name	text	NN
source_row_number	integer	NN
order_id	bigint	NN
user_id	bigint	NN
user_phone	text	NN
address_text	text	NN
created_at	timestamp	NN
paid_at	timestamp	
delivery_started_at	timestamp	
delivered_at	timestamp	
canceled_at	timestamp	
payment_type	text	NN
item_id	bigint	NN
item_title	text	NN
item_category	text	NN
item_quantity	integer	NN
item_price	numeric(12,2)	NN
item_canceled_quantity	integer	NN
item_replaced_id	bigint	
order_discount	numeric(5,2)	NN
item_discount	numeric(5,2)	NN
order_cancellation_reason	text	
driver_id	bigint	NN
driver_phone	text	NN
delivery_cost	numeric(12,2)	NN
store_id	bigint	NN
store_address	text	NN
loaded_at	timestamp	NN

mart.order_metrics_daily		
metric_date	date	NN
metric_year	integer	NN
metric_month	integer	NN
metric_day	integer	NN
city_id	smallint	NN
city_name	text	NN
store_id	bigint	NN
store_name	text	NN
turnover_amount	numeric(14,2)	NN
revenue_amount	numeric(14,2)	NN
expense_amount	numeric(14,2)	NN
profit_amount	numeric(14,2)	NN
created_orders_count	integer	NN
delivered_orders_count	integer	NN
canceled_orders_count	integer	NN
canceled_after_delivery_count	integer	NN
service_error_cancellations_count	integer	NN
buyers_count	integer	NN
average_check	numeric(14,2)	NN
orders_per_buyer	numeric(14,4)	NN
revenue_per_buyer	numeric(14,2)	NN
orders_with_driver_change_count	integer	NN
active_couriers_count	integer	NN
refreshed_at	timestamp	NN

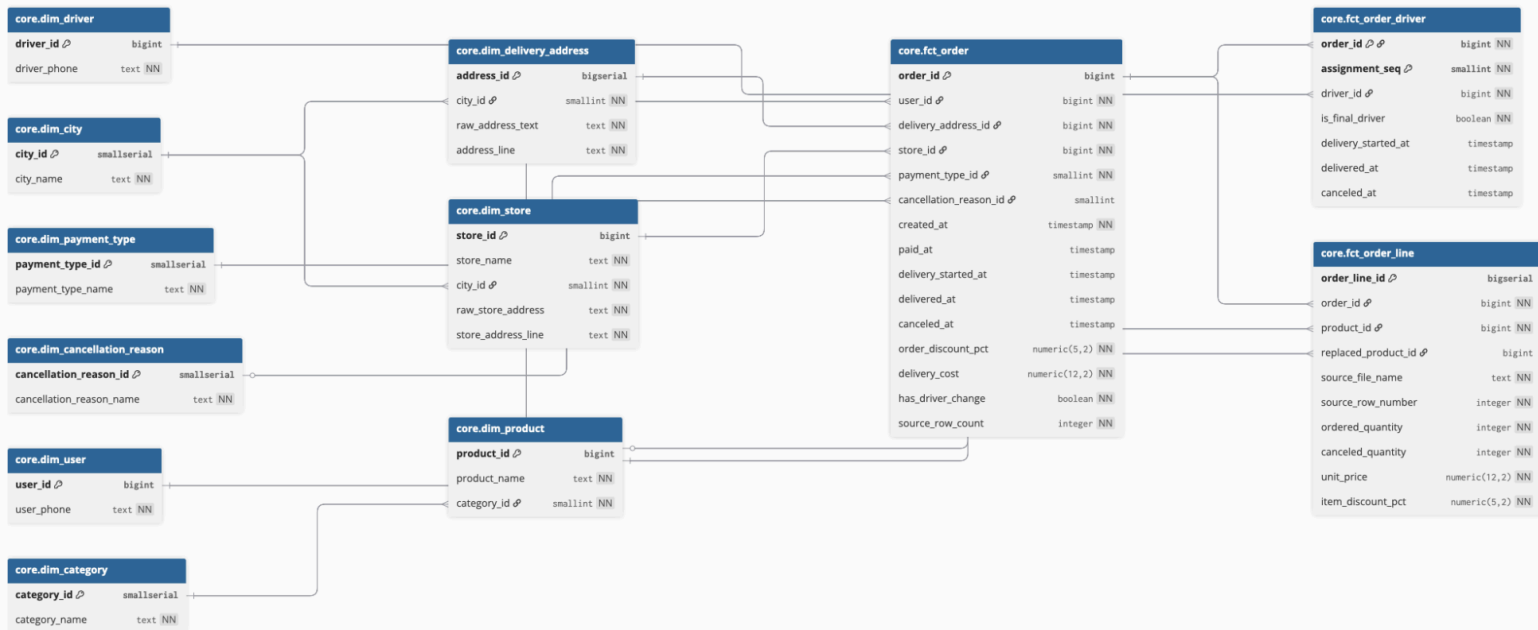


Схема включает три логических слоя:

1. **stage:** Используется для сырой загрузки parquet-файлов без потери исходной структуры.
2. **core:** Содержит нормализованную реляционную модель, построенную в 3NF. Именно этот слой является основной нормализованной схемой проекта.
3. **mart** : Содержит итоговую витрину mart.order_metrics_daily, рассчитанную на основе нормализованных таблиц.

Допущения, принятые в проекте

В процессе проектирования были приняты следующие допущения:

- дата витрины определяется как дата создания заказа;
- оборот рассчитывается по заказанному количеству товара;
- выручка рассчитывается по оплачиваемому количеству;
- оплачиваемое количество определяется как
 $\text{net_quantity} = \text{ordered_quantity} - \text{canceled_quantity}$;
- расходы интерпретируются как delivery_cost для заказов, где доставка началась;
- прибыль рассчитывается как
 $\text{profit} = \text{revenue} - \text{expense}$;
- активные курьеры считаются как количество уникальных driver_id, участвовавших в заказах соответствующего среза.

DDL-скрипты

[sql/00_create_databases.sql](#)

```
CREATE DATABASE airflow OWNER project_user;  
CREATE DATABASE dwh OWNER project_user;
```

sql/10_ddl.sql

```
CREATE SCHEMA IF NOT EXISTS stage;  
CREATE SCHEMA IF NOT EXISTS core;  
CREATE SCHEMA IF NOT EXISTS mart;
```

```
CREATE TABLE IF NOT EXISTS stage.raw_delivery_lines (  
    source_file_name TEXT NOT NULL,  
    source_row_number INTEGER NOT NULL,  
    order_id BIGINT NOT NULL,  
    user_id BIGINT NOT NULL,  
    user_phone TEXT NOT NULL,  
    address_text TEXT NOT NULL,  
    created_at TIMESTAMP NOT NULL,  
    paid_at TIMESTAMP NULL,  
    delivery_started_at TIMESTAMP NULL,  
    delivered_at TIMESTAMP NULL,  
    canceled_at TIMESTAMP NULL,  
    payment_type TEXT NOT NULL,  
    item_id BIGINT NOT NULL,  
    item_title TEXT NOT NULL,  
    item_category TEXT NOT NULL,  
    item_quantity INTEGER NOT NULL,  
    item_price NUMERIC(12, 2) NOT NULL,  
    item_canceled_quantity INTEGER NOT NULL,  
    item_replaced_id BIGINT NULL,  
    order_discount NUMERIC(5, 2) NOT NULL,  
    item_discount NUMERIC(5, 2) NOT NULL,  
    order_cancellation_reason TEXT NULL,  
    driver_id BIGINT NOT NULL,
```

```
driver_phone TEXT NOT NULL,  
delivery_cost NUMERIC(12, 2) NOT NULL,  
store_id BIGINT NOT NULL,  
store_address TEXT NOT NULL,  
loaded_at TIMESTAMP NOT NULL DEFAULT NOW(),  
PRIMARY KEY (source_file_name, source_row_number)  
);
```

```
CREATE TABLE IF NOT EXISTS core.dim_city (  
    city_id SMALLSERIAL PRIMARY KEY,  
    city_name TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE IF NOT EXISTS core.dim_category (  
    category_id SMALLSERIAL PRIMARY KEY,  
    category_name TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE IF NOT EXISTS core.dim_payment_type (  
    payment_type_id SMALLSERIAL PRIMARY KEY,  
    payment_type_name TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE IF NOT EXISTS core.dim_cancellation_reason (  
    cancellation_reason_id SMALLSERIAL PRIMARY KEY,  
    cancellation_reason_name TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE IF NOT EXISTS core.dim_user (  
    user_id BIGINT PRIMARY KEY,  
    user_phone TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE IF NOT EXISTS core.dim_driver (  
    driver_id BIGINT PRIMARY KEY,  
    driver_phone TEXT NOT NULL UNIQUE  
);
```

```
CREATE TABLE IF NOT EXISTS core.dim_delivery_address (  
    address_id BIGSERIAL PRIMARY KEY,  
    city_id SMALLINT NOT NULL REFERENCES core.dim_city(city_id),  
    raw_address_text TEXT NOT NULL UNIQUE,  
    address_line TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS core.dim_store (  
    store_id BIGINT PRIMARY KEY,  
    store_name TEXT NOT NULL,  
    city_id SMALLINT NOT NULL REFERENCES core.dim_city(city_id),  
    raw_store_address TEXT NOT NULL UNIQUE,  
    store_address_line TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS core.dim_product (  
    product_id BIGINT PRIMARY KEY,  
    product_name TEXT NOT NULL,  
    category_id SMALLINT NOT NULL REFERENCES core.dim_category(category_id),  
    UNIQUE (product_name, category_id)  
);
```

```
CREATE TABLE IF NOT EXISTS core.fct_order (  
    order_id BIGINT PRIMARY KEY,  
    user_id BIGINT NOT NULL REFERENCES core.dim_user(user_id),  
    delivery_address_id BIGINT NOT NULL REFERENCES core.dim_delivery_address(address_id),  
    store_id BIGINT NOT NULL REFERENCES core.dim_store(store_id),  
    payment_type_id SMALLINT NOT NULL REFERENCES core.dim_payment_type(payment_type_id),  
    cancellation_reason_id SMALLINT NULL REFERENCES core.dim_cancellation_reason(cancellation_reason_id),  
    created_at TIMESTAMP NOT NULL,  
    paid_at TIMESTAMP NULL,  
    delivery_started_at TIMESTAMP NULL,  
    delivered_at TIMESTAMP NULL,  
    canceled_at TIMESTAMP NULL,  
    order_discount_pct NUMERIC(5, 2) NOT NULL,  
    delivery_cost NUMERIC(12, 2) NOT NULL,  
    has_driver_change BOOLEAN NOT NULL DEFAULT FALSE,  
    source_row_count INTEGER NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS core.fct_order_driver (  
    order_id BIGINT NOT NULL REFERENCES core.fct_order(order_id),  
    assignment_seq SMALLINT NOT NULL,  
    driver_id BIGINT NOT NULL REFERENCES core.dim_driver(driver_id),  
    is_final_driver BOOLEAN NOT NULL,  
    delivery_started_at TIMESTAMP NULL,  
    delivered_at TIMESTAMP NULL,  
    canceled_at TIMESTAMP NULL,  
    PRIMARY KEY (order_id, assignment_seq),  
    UNIQUE (order_id, driver_id)  
);
```

```
CREATE TABLE IF NOT EXISTS core.fct_order_line (  
    order_line_id BIGSERIAL PRIMARY KEY,  
    order_id BIGINT NOT NULL REFERENCES core.fct_order(order_id),  
    product_id BIGINT NOT NULL REFERENCES core.dim_product(product_id),  
    replaced_product_id BIGINT NULL REFERENCES core.dim_product(product_id),  
    source_file_name TEXT NOT NULL,  
    source_row_number INTEGER NOT NULL,  
    ordered_quantity INTEGER NOT NULL CHECK (ordered_quantity >= 0),  
    canceled_quantity INTEGER NOT NULL CHECK (canceled_quantity >= 0 AND canceled_quantity <= ordered_quantity),  
    unit_price NUMERIC(12, 2) NOT NULL CHECK (unit_price >= 0),  
    item_discount_pct NUMERIC(5, 2) NOT NULL CHECK (item_discount_pct >= 0),  
    UNIQUE (source_file_name, source_row_number)  
);
```

```
CREATE INDEX IF NOT EXISTS idx_stage_raw_order_id  
    ON stage.raw_delivery_lines(order_id);
```

```
CREATE INDEX IF NOT EXISTS idx_core_order_created_at  
    ON core.fct_order(created_at);
```

```
CREATE INDEX IF NOT EXISTS idx_core_order_store_id  
    ON core.fct_order(store_id);
```

```
CREATE INDEX IF NOT EXISTS idx_core_order_line_order_id  
    ON core.fct_order_line(order_id);
```

```
CREATE INDEX IF NOT EXISTS idx_core_order_line_product_id  
    ON core.fct_order_line(product_id);
```

```
CREATE TABLE IF NOT EXISTS mart.order_metrics_daily (  
    metric_date DATE NOT NULL,  
    metric_year INTEGER NOT NULL,  
    metric_month INTEGER NOT NULL,  
    metric_day INTEGER NOT NULL,  
    city_id SMALLINT NOT NULL,  
    city_name TEXT NOT NULL,  
    store_id BIGINT NOT NULL,  
    store_name TEXT NOT NULL,  
    turnover_amount NUMERIC(14, 2) NOT NULL,  
    revenue_amount NUMERIC(14, 2) NOT NULL,  
    expense_amount NUMERIC(14, 2) NOT NULL,  
    profit_amount NUMERIC(14, 2) NOT NULL,  
    created_orders_count INTEGER NOT NULL,  
    delivered_orders_count INTEGER NOT NULL,  
    canceled_orders_count INTEGER NOT NULL,  
    canceled_after_delivery_count INTEGER NOT NULL,  
    service_error_cancellations_count INTEGER NOT NULL,  
    buyers_count INTEGER NOT NULL,  
    average_check NUMERIC(14, 2) NOT NULL,  
    orders_per_buyer NUMERIC(14, 4) NOT NULL,  
    revenue_per_buyer NUMERIC(14, 2) NOT NULL,  
    orders_with_driver_change_count INTEGER NOT NULL,  
    active_couriers_count INTEGER NOT NULL,  
    refreshed_at TIMESTAMP NOT NULL DEFAULT NOW(),  
    PRIMARY KEY (metric_date, store_id)  
);
```